TASK S4.01

SQL Modeling

Level 1

Descàrrega els arxius CSV, estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:

Database ‘starmarketplace’

Database design

The database will have the name 'starmarketplace', as the statement asks I will build the DB from the CSVs that are provided to us. As in the previous exercise I will consider that it is a database of a marketplace. We are asked to design the DB with a star scheme that contains at least 4 tables. Analyzing the CSV files I see that the fact table, the central DB is 'transactions' since analyzed in detail is the one that has the greatest cardinality and also the one that contains the Foreign Key (card_id, business_id, product_ids and user_id) of the rest of the tables, although we have not yet declared the relationships.

The tables (companies, credit_cards, products, european_users, american_users) are the dimension tables of this DB. In these tables, which all have a much lower cardinality , we find fields that offer characteristics about the metrics contained in the table of facts ‘transactions'.

For this exercise of creation and design of the DB, and import of the CSV data I will use a ETL method and I will proceed following this steps:

1. Create the DB.

2. Table creation: I will create tables for each CSV without many restrictions and wide margin in the DataType, neither I will designate PK or FK at the starting moment: So I can make an indiscriminate import without risk of errors.

   The 'european_users' and 'american-users' tables contain the same type of data and structures, the rational thing is to unify them under a single 'users' table. The fact that the data in these 2 tables are separate by continent, gives us in itself valuable information that we would lose if we limited ourselves to joining them. That is why it is very important to add to the new table a field that I will call 'continent' that will give us the information of whether the customers are European or American.

3. Data import from CSV to DB tables.

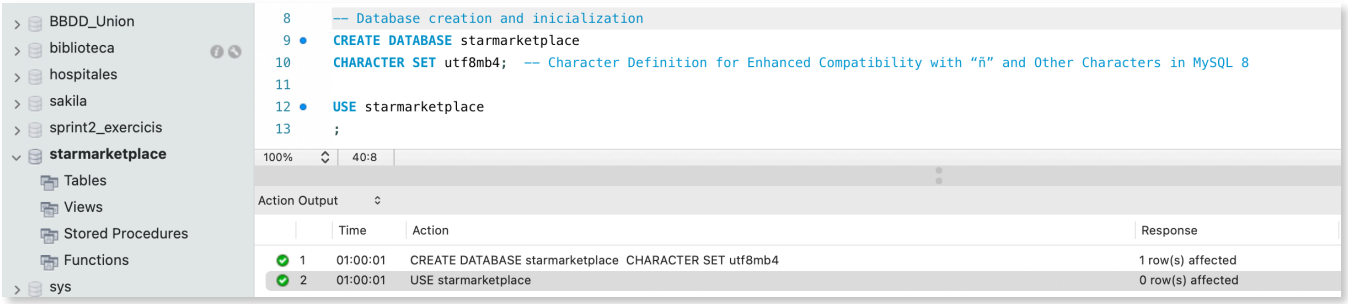
4. Data Cleaning & Data Transforming:
   I will debug and analyze the tables by adjusting the appropriate DataType and data-length for each field of each table.

5. Data modeling: We will create the relationships between the table of facts and those of dimensions.

6. At the starting moment we will also load the 'products' table & its data to the DB, although we will not yet create the relationships with 'transactions' since it is requested at level 3. Anyway, I have already detected that the relationship between 'transactions' and 'products' is from many to many, in each transaction several products can be purchased and each product can be sold many times (or appear in many transactions). At the appropriate time I will use an intermediate table to create the relationship and convert the relationship ('n' to 'n') into a relationship ('1 a n' vs 'n a 1').

Step 1 - Database creation and initialization

Character definition for enhanced compatibility with “ñ” and other characters in MySQL 8.

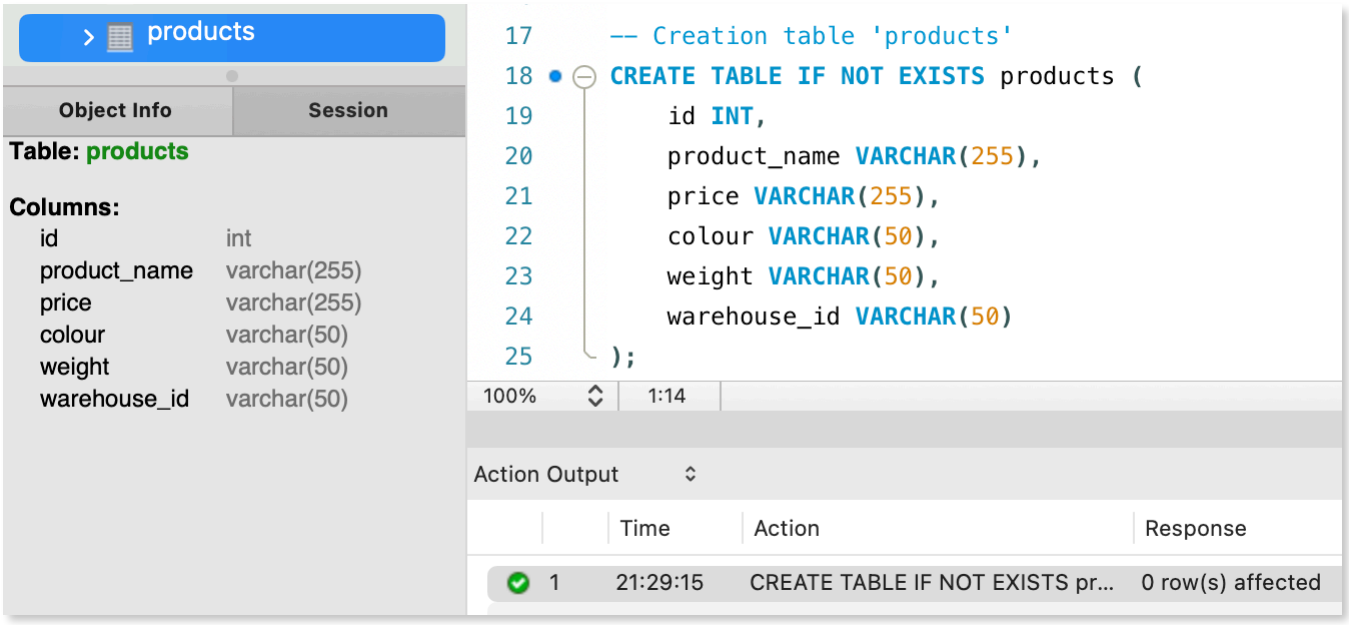


Step 2 - Table creation:

As a general rule I will create all the fields of all the tables with DataType VARCHAR with this I will avoid errors in the data loading. Later I will adjust the DataType to its real data type. The PK fields, mainly 'id' if its possible I will put them as INT and the other numerical fields I will put it INT, DECIMAL or FLOAT as appropriate or DATE or TIMESTAMP if is date. I also would like to comment that I incorporate the clause “IF NOT EXIST” as a good praxis to avoid any error in case that the table already exists.

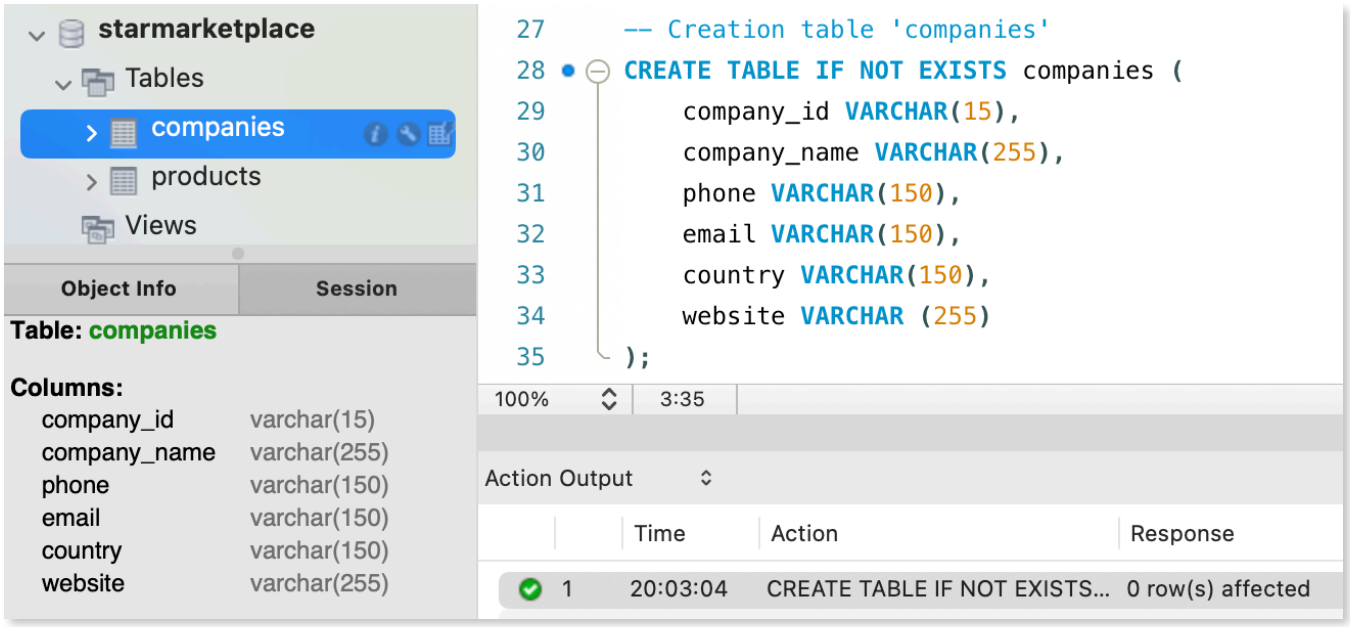
‘products’ table - dimension table:

- This table allows us to create the 'id' as INT
- Important to highlight the 'price' field we must load as VARCHAR since the $ symbol makes it impossible to load as a DECIMAL field



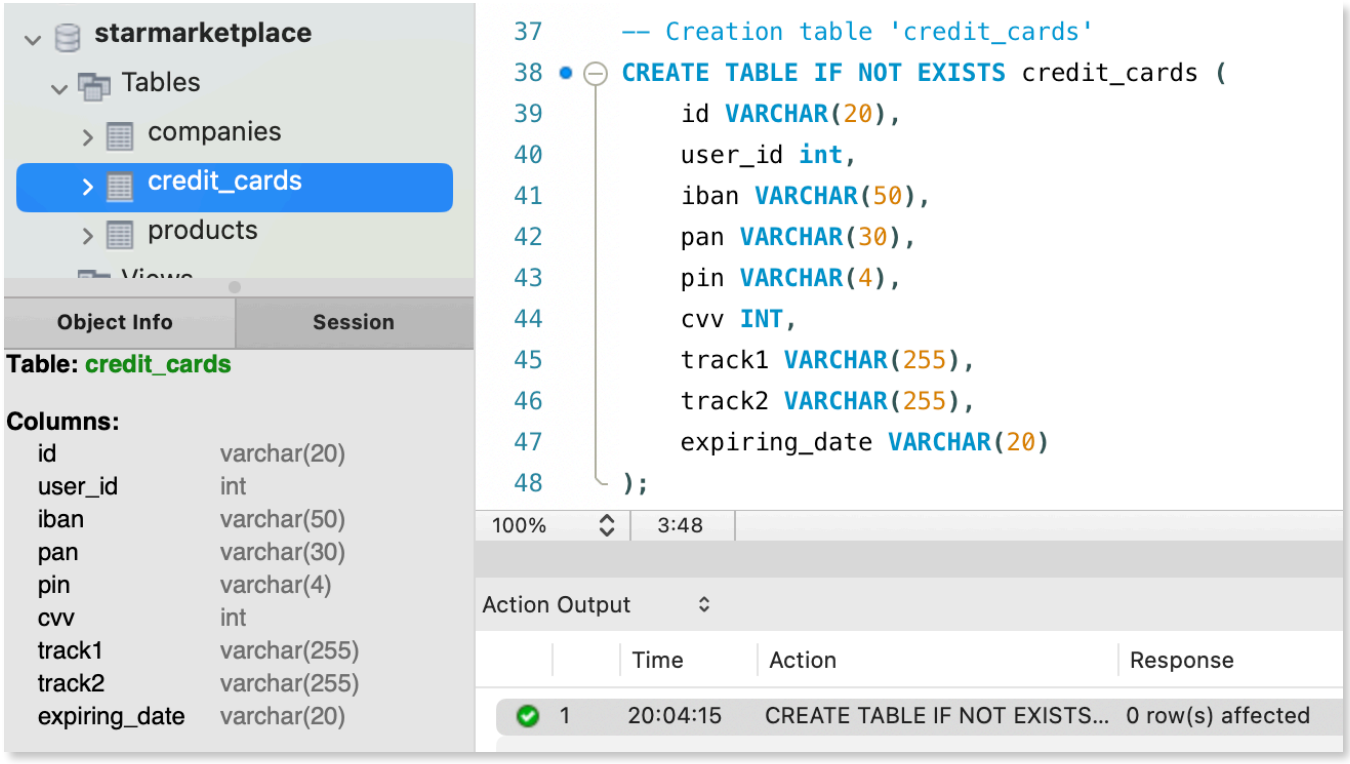
‘companies’ table - dimension table:

- I have to create the ‘company_id’ column as a VARCHAR because is alphanumerical.
- The rest will be also VARCHAR.



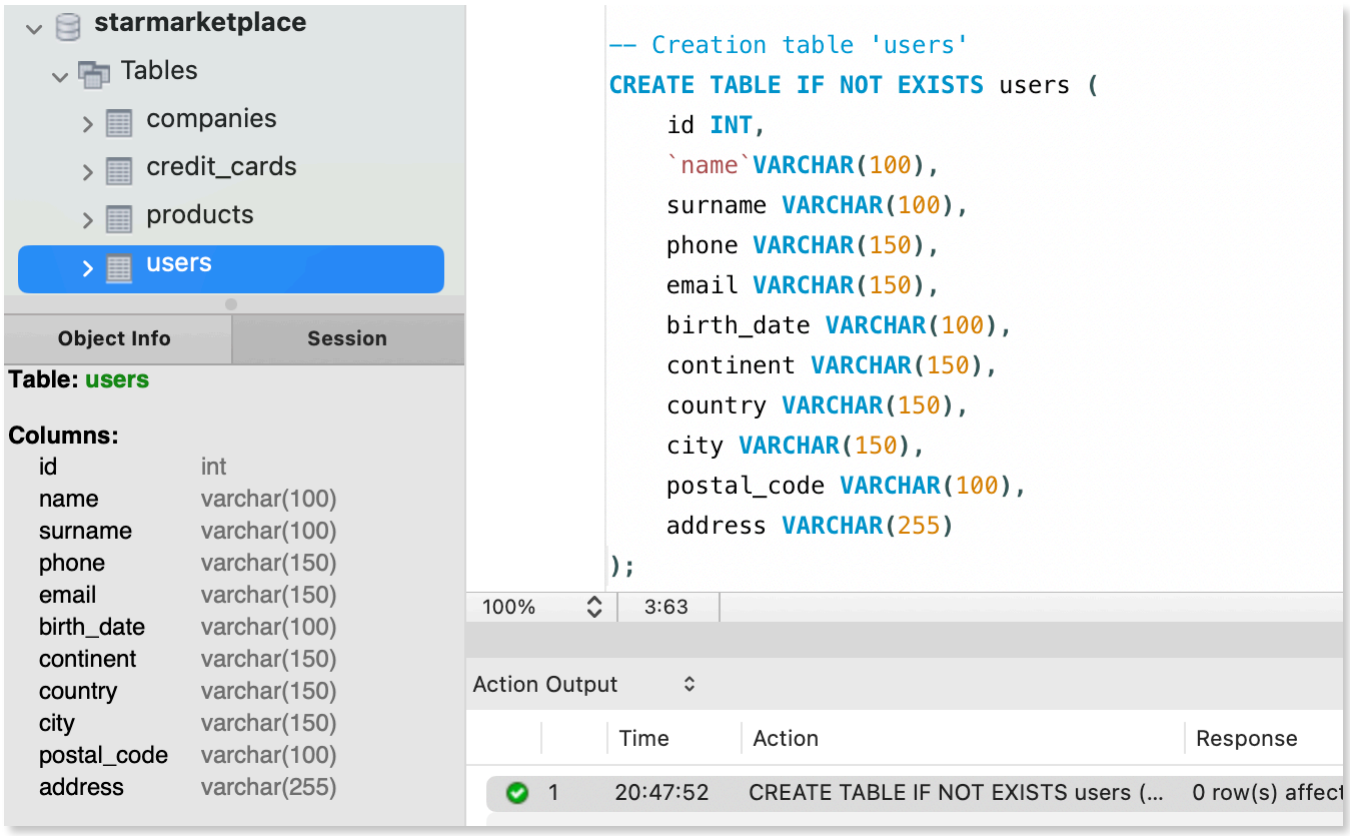
‘credit_cards’ table - dimension table:

- I have to create the ‘company_id’ column as a VARCHAR because is alphanumerical.
- The rest will be also VARCHAR. Except ‘user_id’ & ’cvv’ that is clear an INT.



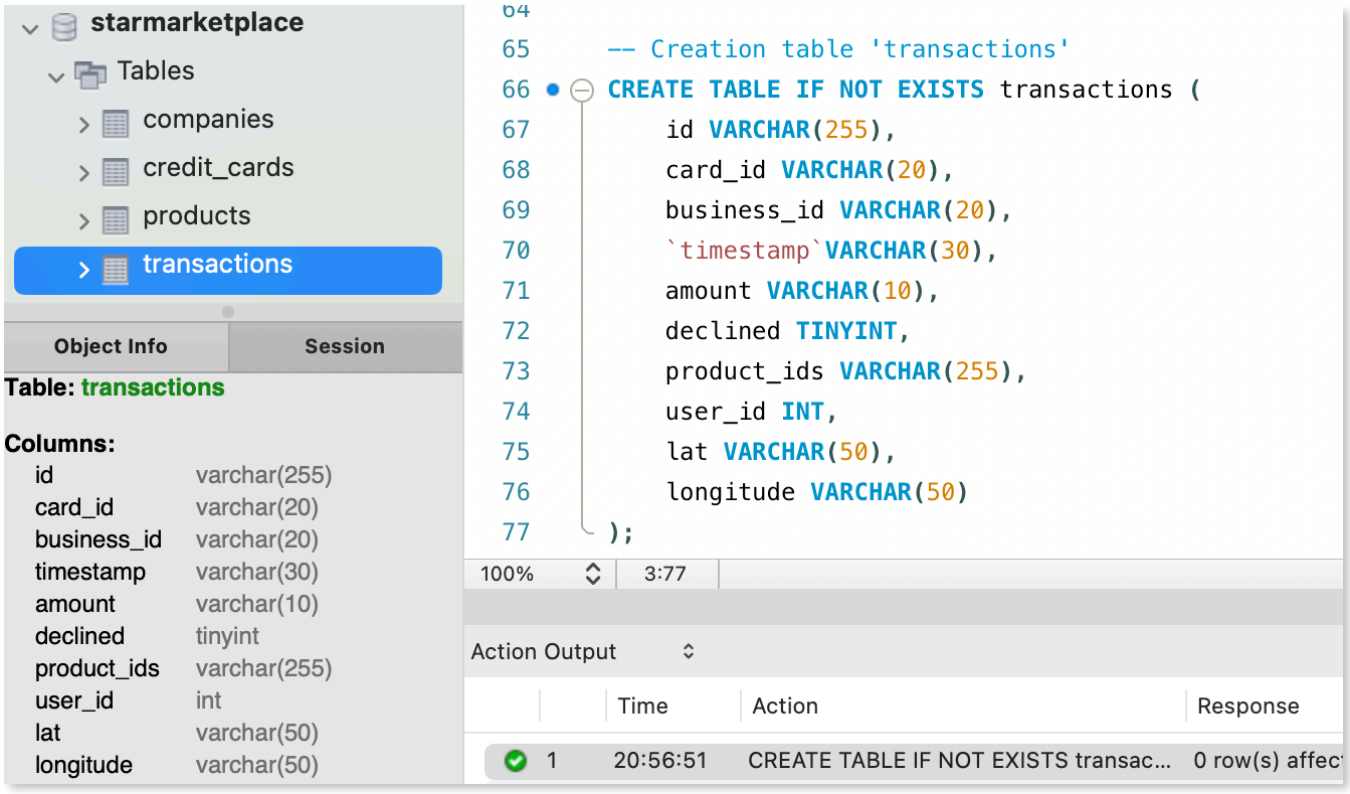
‘users’ table - dimension table:

- This point is very important since here I have made the decision to create a single 'users' table that will receive the data of the 'european_users' and 'american_users' tables.
- The justification: they are two tables of identical structure and type of content.
- The problem: if we join the two tables into one without making changes, we would lose relevant information that is the belonging of a country to a continent or another.
- The solution: creation of a new 'continent' field in this 'users' table that, when we import, it will receive the information of the continent to which each country belongs.
- We created the 'id' field as INT since clearly all the data is of this type.
- We create the rest of the fields VARCHAR to avoid errors, later we will convert 'birth-date' to the appropriate type.



‘transactions’ table - fact table:

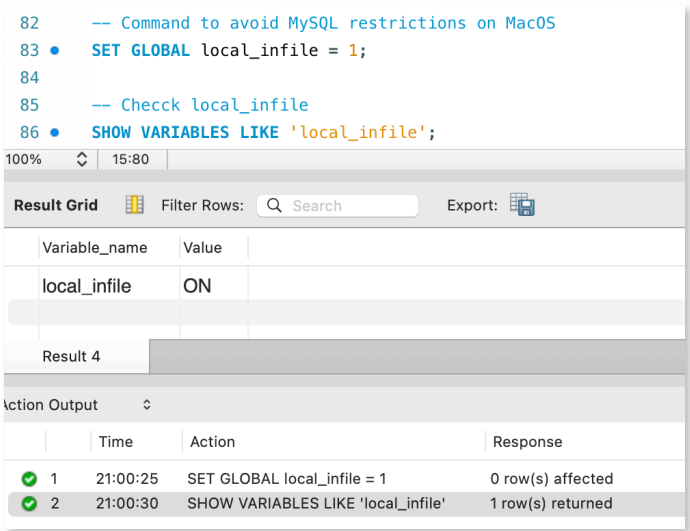
- This is the table of facts. It is the center of the star we want to create.
- It contains the FK and a large number of rows or records, with more possibility of having errors when loading data, for this reason we will use VARCHAR in most columns.
- 'Id' I created it as VARCHAR and later we will find if there are duplicates or errors.
- 'Declined' as it is 0 or 1 I think it as TINYINT
- 'User_id' is clearly an INT



Step 3 - Data import

First of all, comment that working with MacOS I have encountered a problem when loading data related to the protection of the MySQL server to data loading.

- Error Code: 1290. The MySQL server is running with the --secure-file-priv option
- I have investigated it and there could be several causes and possible solutions. In principle there is a restriction of the MySQL server that limits the import only from a specific directory of the system: /usr/local/mysql/data/
- An easy solution was to copy the CSVs to this directory, but I have opted for an alternative solution that is to modify the configuration of MySQL Workbench, which in my case was also exercising a protection / limitation.
- I have edited the connection to the MySQL server Workbench settings/ advance/others : OPT_LOCAL_INFILE=1
- I have to run the following statement to tell Workbench that the files will be in local: SET GLOBAL local_infile = 1
- Finally because of this, now the LOAD call to import will be on local: LOAD DATA LOCALError Code: 1290.
  The MySQL server is running with the --secure-file-priv option.



Below I present the import of tables:

- All imports have been processed correctly with the table design that I have established in the previous point.
- I first present the statement with its result and then a small sample of rows of a SELECT ALL.
- In general: the fields are separated by commas, text by quotation marks and all have a header row
- In certain tables I will comment on specific cases.

Import data to 'products':

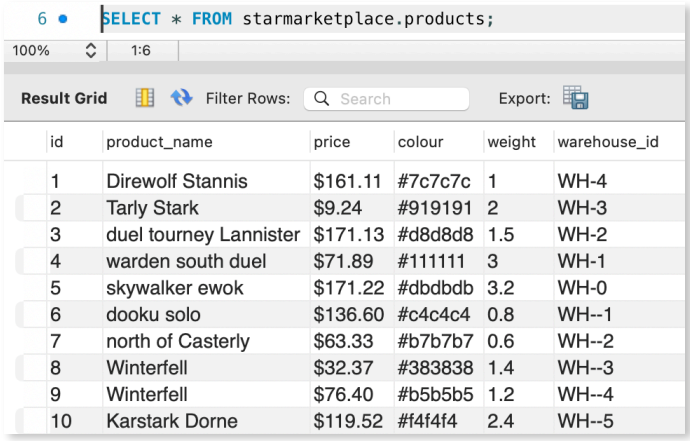
-Total 100 rows imported

```
88    -- Import CSV data to 'products'
89    LOAD DATA LOCAL
90    INFILE '/Users/rogerdefez/Documents/Cursos i Llibres/BootCamp IT Academy/03_Especialitzacio/Sprint_4_Modelat_SQL/Dades_originals_S4/products.csv'
91    INTO TABLE products
92    FIELDS TERMINATED BY ','
93    ENCLOSED BY '"'
94    IGNORE 1 ROWS
95    ;
```

100% 1:79

Action Output

	Time	Action	Response	Duration / F
1	21:11:28	LOAD DATA LOCAL INFILE '/Users/roge...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.033 sec



```
6    SELECT * FROM starmarketplace.products;
```

100% 1:6

Result Grid Filter Rows: Search Export:

id	product_name	price	colour	weight	warehouse_id
1	Direwolf Stannis	$161.11	#7c7c7c	1	WH-4
2	Tarly Stark	$9.24	#919191	2	WH-3
3	duel tourney Lannister	$171.13	#d8d8d8	1.5	WH-2
4	warden south duel	$71.89	#111111	3	WH-1
5	skywalker ewok	$171.22	#dbdbdb	3.2	WH-0
6	dooku solo	$136.60	#c4c4c4	0.8	WH--1
7	north of Casterly	$63.33	#b7b7b7	0.6	WH--2
8	Winterfell	$32.37	#383838	1.4	WH--3
9	Winterfell	$76.40	#b5b5b5	1.2	WH--4
10	Karstark Dorne	$119.52	#f4f4f4	2.4	WH--5

Import data to 'companies':

-Total 100 rows imported

```
97    -- Import CSV data to 'companies'
98 •  LOAD DATA LOCAL
99    INFILE '/Users/rogerdefez/Documents/Cursos i Llibres/BootCamp IT Academy/03_Especialitzacio/Sprint_4_Modelat_SQL/Dades_originals_S4/companies.csv'
100   INTO TABLE companies
101   FIELDS TERMINATED BY ','
102   ENCLOSED BY '"'
103   IGNORE 1 ROWS
104   ;
```

100% 14:94

Action Output

	Time	Action	Response	Duration / F
1	21:25:14	LOAD DATA LOCAL INFILE '/Users/roge...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.055 sec

```
6 •  SELECT * FROM starmarketplace.companies
7    LIMIT 10;
```

100% 10:7

Result Grid Filter Rows: Search Export:

company_id	company_name	phone	email	country	website
b-2222	Ac Fermentum Incorporated	06 85 56 52 33	donec.porttitor.tellus@yahoo.net	Germany	https://instagram.com/site
b-2226	Magna A Neque Industries	04 14 44 64 62	risus.donec.nibh@icloud.org	Australia	https://whatsapp.com/group/9
b-2230	Fusce Corp.	08 14 97 58 85	risus@protonmail.edu	United States	https://pinterest.com/sub/cars
b-2234	Convallis In Incorporated	06 66 57 29 50	mauris.ut@aol.couk	Germany	https://cnn.com/user/110
b-2238	Ante Iaculis Nec Foundation	08 23 04 99 53	sed.dictum.proin@outlook.ca	New Zealand	https://netflix.com/settings
b-2242	Donec Ltd	01 25 51 37 37	at.iaculis@hotmail.couk	Norway	https://nytimes.com/user/110
b-2246	Sed Nunc Ltd	02 62 64 73 48	nibh@yahoo.org	United Kingdom	https://cnn.com/one
b-2250	Amet Nulla Donec Corporation	07 15 25 14 74	mattis.integer.eu@protonmail.net	Italy	https://netflix.com/sub/cars
b-2254	Nascetur Ridiculus Mus Inc.	06 26 87 61 84	suspendisse.dui@icloud.net	United States	https://ebay.com/sub
b-2258	Vestibulum Lorem PC	02 02 87 33 40	aenean.massa.integer@aol.net	Belgium	https://pinterest.com/sub/cars

Import data to 'credit_cards':

- Total 5000 rows imported

```
106   -- Import CSV data to 'credit_cards'
107 •  LOAD DATA LOCAL
108   INFILE '/Users/rogerdefez/Documents/Cursos i Llibres/BootCamp IT Academy/03_Especialitzacio/Sprint_4_Modelat_SQL/Dades_originals_S4/credit_cards.csv'
109   INTO TABLE credit_cards
110   FIELDS TERMINATED BY ','
111   ENCLOSED BY '"'
112   IGNORE 1 ROWS
113   ;
```

100% 16:102

Action Output

	Time	Action	Response	Duration / Fetch
1	21:26:04	LOAD DATA LOCAL INFILE '/Users/roge...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.054 sec

```
6 •  SELECT * FROM starmarketplace.credit_cards
7    LIMIT 10;
```

100% 1:4

Result Grid Filter Rows: Search Export:

id	user_id	iban	pan	pin	cvv	track1	track2	expiring_date
CcU-2938	275	TR301950312213576817…	5424465566813633	3257	984	%B8383712448554646^Wovsx…	%B7653863056044187=800716333…	10/30/22
CcU-2945	274	DO26854763748537475 21…	5142423821948828	9080	887	%B4621311609958661^Uftuyfs…	%B4149568437843501=510714033…	08/24/23
CcU-2952	273	BG45IVQL52710525608255	4556 453 55 5287	4598	438	%B2183285104307501^Cddyyt…	%B6778580257827162=690685974…	06/29/21
CcU-2959	272	CR7242477244335841535	372461377349375	3583	667	%B7281111956795320^Xocddij…	%B4246154489281853=280522391…	02/24/23
CcU-2966	271	BG72LKTQ15627628377363	448566 886747 7265	4900	130	%B4728932322756223^Jhlgvs…	%B2318571115599881=890821578 4?5	10/29/24
CcU-2973	270	PT8780622813509242945…	544 58654 54343 384	8760	887	%B4761405253275637^Hjnnip…	%B7816169831446746=1310277 2?9	01/30/25
CcU-2980	269	DE39241881883086277136	402400 7145845969	5075	596	%B7320483593870549^Ookzq…	%B2474313962214151=041221913…	07/24/22
CcU-2987	268	GE89681434837748781813	3763 747687 76666	2298	797	%B4750646345146674^Pjmlyrf…	%B5441935173418615=410370453…	10/31/23
CcU-2994	267	BH62714428368066765294	344283273252593	7545	595	%B1583759784015674^Gmqo…	%B4141467473024349=650680095…	02/28/22
CcU-3001	266	CY4908742665477458126…	511722 924833 2244	9562	867	%B6227288756728648^Awxilfc…	%B3429355750963453=530526830…	09/16/22

Import data to ‘users’:

- Total 5000 rows imported, 3990 from ‘european_users’ & 1010 from ‘american_users’.
- In this import I have taken advantage of the possibility of entering a fixed value in one of the fields at the time of importing a CSV that lacks that field. Use for it SET continent = “xxxx”. In these cases it is important to include the fields from the CSV (id, name, surname, phone, email, birth_date, country, city, postal_code, address) so that the import locates each data in its position.
- I take advantage of the 'continent' field to preserve the relevant geographic area information while reducing the size of the DB and I can keep my design in star.

```
115    -- Import 'european_users' CSV data to 'users'
116 •  LOAD DATA LOCAL
117    INFILE '/Users/rogerdefez/Documents/Cursos i Llibres/BootCamp IT Academy/03_Especialitzacio/Sprint_4_Modelat_SQL/Dades_originals_S4/european_users.csv'
118    INTO TABLE users
119    FIELDS TERMINATED BY ','
120    ENCLOSED BY '"'
121    IGNORE 1 ROWS
122    (id, `name`, surname, phone, email, birth_date, country, city, postal_code, address)
123    SET continent = "Europe"
124    ;
```

100% 16:111

Action Output

	Time	Action	Response	Duration / Fetch Time
1	21:27:53	LOAD DATA LOCAL INFILE '/Users/roge...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.050 sec

```
126    -- Import 'american_users' CSV data to 'users'
127 •  LOAD DATA LOCAL
128    INFILE '/Users/rogerdefez/Documents/Cursos i Llibres/BootCamp IT Academy/03_Especialitzacio/Sprint_4_Modelat_SQL/Dades_originals_S4/american_users.csv'
129    INTO TABLE users
130    FIELDS TERMINATED BY ','
131    ENCLOSED BY '"'
132    IGNORE 1 ROWS
133    (id, `name`, surname, phone, email, birth_date, country, city, postal_code, address)
134    SET continent = "America"
135    ;
```

100% 25:123

Action Output

	Time	Action	Response	Duration / Fetch Time
1	21:29:28	LOAD DATA LOCAL INFILE '/Users/roge...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.017 sec

```
6 •  SELECT * FROM starmarketplace.users;
```

100% 1:3

Result Grid Filter Rows: Search Export: Fetch rows:

id	name	surname	phone	email	birth_date	continent	country	city	postal_code	address
147	Brody	Talley	(307) 307-2751	metus.sit.amet@outlook.org	Jun 13, 1991	America	United States	Houston	77001	469-5852 Tellus Street
148	Kerry	Adkins	(528) 872-1974	augue.eu.tempor@icloud.couk	Mar 13, 1983	America	United States	Chicago	60601	Ap #422-4836 Nunc Rd.
149	Brock	Doyle	(265) 140-9567	cursus.a@aol.edu	Feb 19, 1986	America	United States	Chicago	60601	Ap #172-5737 Lorem St.
150	Oleg	Coleman	1-131-139-5673	dis@outlook.edu	Dec 2, 1981	America	United States	Chicago	60601	722-3056 Eu
151	Meghan	Hayden	0800 746 6747	arcu.vel@hotmail.ca	Jul 2, 1980	Europe	United Kingdom	London	EC1A 1BB	Ap #432-4493 Aliquet Rd.
152	Hakeem	Alford	(0111) 367 0184	adipiscing.ligula@google.edu	Sep 30, 1979	Europe	United Kingdom	Birmingham	B1 1AA	551-8930 Lobortis Street
153	Keegan	Pugh	(016977) 3851	sodales.nisi@aol.org	Jul 27, 1994	Europe	United Kingdom	London	EC1A 1BB	Ap #312-5898 Consecte..
154	Cooper	Bullock	(021) 2521 6627	et@outlook.net	Nov 2, 1986	Europe	United Kingdom	Manchester	M1 1AE	872-1866 Pede Rd.
155	Joshua	Russell	055 4409 5286	justo.nec.ante@outlook.edu	Jan 23, 1984	Europe	United Kingdom	Manchester	M1 1AE	Ap #285-4727 Auctor. Av.
156	Remedios	Case	055 3114 1566	mollis.phasellus.libero@aol.com	Oct 9, 1994	Europe	United Kingdom	Liverpool	L1 1AA	479-3690 Turpis Road

Import data to ‘transactions’:

- Total 100,000 rows imported, due to its large volume of rows it is a delicate process and where errors can appear, we have minimized it thanks to the DataType in the creation.
- Important for this CSV, the separation for field is the "; ", since in the products_ids column the comma is used to separate the ‘product_ids'. If we use the comma as in other tables, the load would give an error.
- This table of facts it will contains the FKs to create the relationships and also the measures that will allow us to make calculations and conclusions about the characteristics of the dimension tables.

```
137    -- Import CSV data to 'transactions'
138    LOAD DATA LOCAL
139    INFILE '/Users/rogerdefez/Documents/Cursos i Llibres/BootCamp IT Academy/03_Especialitzacio/Sprint_4_Modelat_SQL/Dades_originals_S4/transactions.csv'
140    INTO TABLE transactions
141    FIELDS TERMINATED BY ';'
142    ENCLOSED BY '"'
143    IGNORE 1 ROWS
144    ;
```

100% 2:135

Action Output

	Time	Action	Response	Duration / Fetch
1	21:33:21	LOAD DATA LOCAL INFILE '/Users/roge...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0	0.367 sec

```
6    SELECT * FROM starmarketplace.transactions
7    LIMIT 10;
```

100% 1:4

Result Grid Filter Rows: Search Export:

id	card_id	business...	timestamp	amount	declined	product_ids	user_id	lat	longitude
CDDA7E40-544D-47BB-A4ED-671DD8A950D9	CcS-6894	b-2466	2018-12-12 08:05:17	161.88	0	75, 73, 98	2313	59.620509743...	16.55997715...
09456357-8E9B-475A-8257-87A023699964	CcS-5135	b-2342	2024-05-20 22:49:08	171.13	0	3	554	45.764588419...	4.843056518...
C47C7C84-C174-4973-A76B-825A9DB5582A	CcS-8415	b-2250	2018-11-04 23:16:18	497.29	0	92, 85, 36, 23	3834	52.068496084...	4.301099382...
2FB526AA-3844-4DDF-AC09-2EBCC255EE76	CcS-6553	b-2610	2022-06-17 09:17:25	344.15	0	5, 27	1972	39.475985131...	-0.376419443...
0D877407-B85E-40D2-9229-6BD6F2B8DF0A	CcS-7678	b-2454	2020-11-29 07:29:16	435.1	0	38, 32, 82, 62	3097	51.436569850...	5.478525648...
A0BD09EE-7199-41CF-9CB5-3205509FA02F	CcS-6416	b-2438	2021-05-24 22:28:26	364.11	0	92, 64	1835	51.495191890...	18.88912480...
29322BF7-84F4-4A6C-9FFC-C60B6BBD8DCD	CcS-5036	b-2222	2022-05-19 04:50:50	678.61	0	6, 12, 100, 22, 23	455	51.609077736...	19.16003875...
3F614E9B-1EEC-4FB7-8B88-ABCD6C519EED	CcS-8216	b-2526	2019-08-18 21:09:25	207.52	0	69, 97, 31	3635	33.458167495...	-112.0828957...
133534F1-81E5-4688-B4AF-54E516504E9E	CcS-7160	b-2614	2021-12-23 23:58:33	368.64	0	51, 3, 68	2579	60.481217507...	18.81002401...
185D9EC6-9540-4E4F-B38D-395B26C467AC	CcS-5819	b-2550	2019-03-23 19:42:15	57.25	0	69	1238	42.145406999...	12.21245353...

Step 4 - Data cleaning & data transformation

In this step we will work with the data and tables. I will go table by table correcting data errors, changing DataType to fit the content, I will declare the PK and other processes I will follow these steps:

- Exploratory data analysis (EDA).
- Creation of primary keys.
- Normalization of data types.

‘Products’ table:

- Primary Key —> ‘id’.
- Fix 'price' by eliminating $ and changing type to DECIMAL.
- Fix 'weight' change type to DECIMAL.

```
172    -- Transformation
173    ALTER TABLE products
174    ADD PRIMARY KEY (id)
175    ;
```

100% 23:164

Action Output

	Time	Action	Response
5	20:47:46	ALTER TABLE products ADD PRIMARY KEY (id)	0 row(s) affe

```
177    UPDATE products
178    SET price = REPLACE(price, '$', '')
179    WHERE id >= 1
180    ;
181    ALTER TABLE products
182    MODIFY price DECIMAL(10, 2)
183    ;
184    ALTER TABLE products
185    MODIFY weight DECIMAL(5, 1)
186    ;
```

100% 21:173

Action Output

	Time	Action	Response
1	21:39:46	UPDATE products SET price = REPLACE(price, '$', '') WHERE id >= 1	100 row(s) affected Rows matched: 100 Changed: 100 Wa
2	21:39:49	ALTER TABLE products MODIFY price DECIMAL(10, 2)	100 row(s) affected Records: 100 Duplicates: 0 Warnings:
3	21:39:54	ALTER TABLE products MODIFY weight DECIMAL(5, 1)	100 row(s) affected Records: 100 Duplicates: 0 Warnings:

‘companies’ table:

- Primary Key —> ‘company_id’.
- It is not necessary to change types, neither of the PK nor of the rest of the fields.
- Using EDA analysis, I have verified that this table has no duplicates or anything remarkable, just to mention that apparently by numbering the company_id it seems that there is only a part of the companies in this dataset.

```
188    -- Cleaning & transforming 'companies' table
189
190    SELECT COUNT(*) FROM companies;                          -- 100 rows
191    SELECT DISTINCT company_id FROM companies;               -- 100 diferent id (this is OK) No duplicate
192    SELECT DISTINCT company_name FROM companies;             -- 100 diferent company_name (this is OK)  No duplicate
193    SELECT MIN(company_id), MAX(company_id) FROM companies;
194    SELECT * FROM companies WHERE company_name IS NULL;
195
```

100% 18:196

Action Output

	Time	Action	Response
6	10:35:07	SELECT COUNT(*) FROM companies	1 row(s) returned
7	10:35:19	SELECT DISTINCT company_id FROM companies	100 row(s) returned
8	10:35:59	SELECT DISTINCT company_name FROM companies	100 row(s) returned
9	10:36:48	SELECT MIN(company_id), MAX(company_id) FROM companies	1 row(s) returned
10	10:37:26	SELECT * FROM companies WHERE company_id IS NULL	0 row(s) returned
11	10:37:35	SELECT * FROM companies WHERE company_name IS NULL	0 row(s) returned

```
196    -- Transformation
197    ALTER TABLE companies
198    ADD PRIMARY KEY (company_id)
199    ;
```

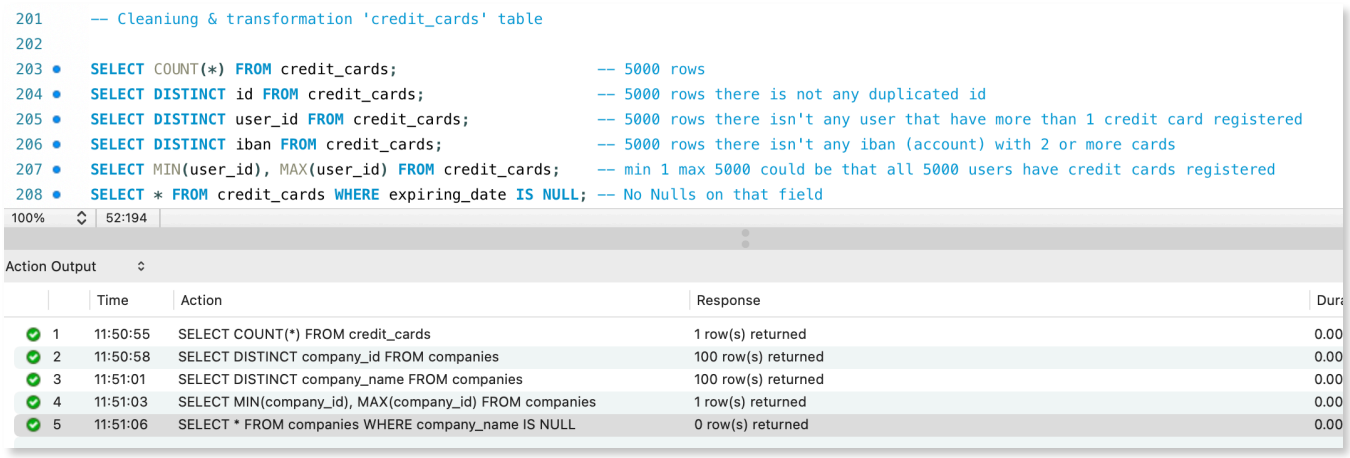
100% 2:185

Action Output

	Time	Action	Response
1	10:55:40	ALTER TABLE companies ADD PRIMARY KEY (company_id)	0 row(s) a

‘credit_cards’ table:

- Primary Key —> ‘id’.
- It is not necessary to change types of the PK (id) or the rest of the fields except 'expiring_date, which I will explain below.
- Using EDA analysis, I have verified that this table has no duplicates or anything remarkable about it.
- As a curious fact after the pre-analysis, the number of unique cards is 5000 equivalent to the number of unique 'user_id' and the unique 'iban' number. This indicates that each user has only one card registered and that there is no account (iban) with 2 or more cards.

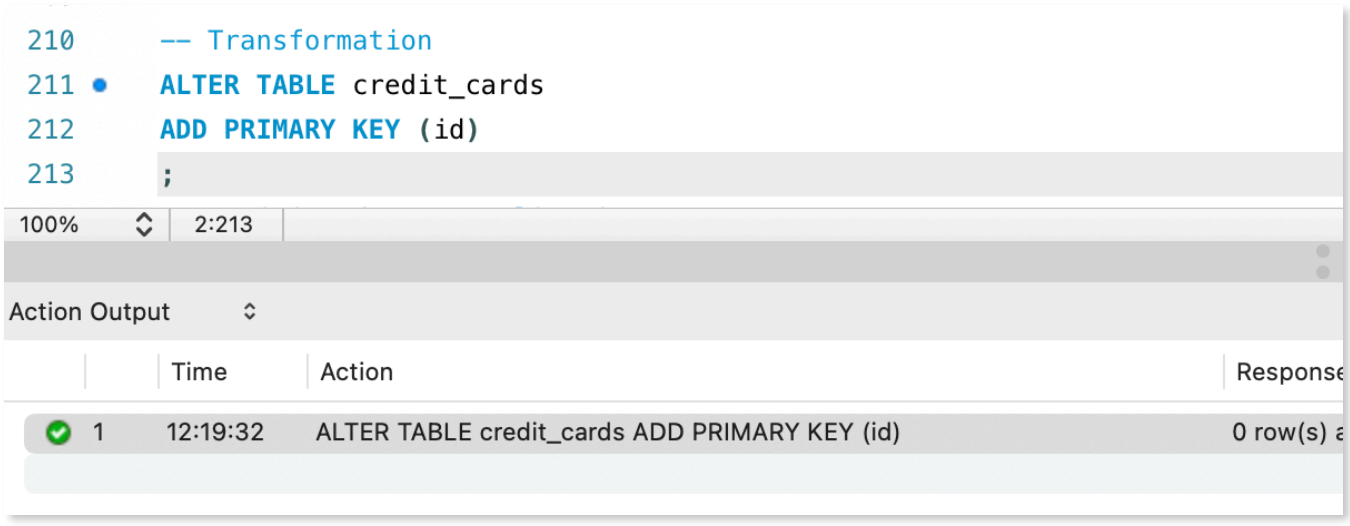


```
201  -- Cleaniung & transformation 'credit_cards' table
202
203  SELECT COUNT(*) FROM credit_cards;                    -- 5000 rows
204  SELECT DISTINCT id FROM credit_cards;                 -- 5000 rows there is not any duplicated id
205  SELECT DISTINCT user_id FROM credit_cards;            -- 5000 rows there isn't any user that have more than 1 credit card registered
206  SELECT DISTINCT iban FROM credit_cards;               -- 5000 rows there isn't any iban (account) with 2 or more cards
207  SELECT MIN(user_id), MAX(user_id) FROM credit_cards;  -- min 1 max 5000 could be that all 5000 users have credit cards registered
208  SELECT * FROM credit_cards WHERE expiring_date IS NULL; -- No Nulls on that field
```

100% 52:194

Action Output

	Time	Action	Response	Dura
1	11:50:55	SELECT COUNT(*) FROM credit_cards	1 row(s) returned	0.00
2	11:50:58	SELECT DISTINCT company_id FROM companies	100 row(s) returned	0.00
3	11:51:01	SELECT DISTINCT company_name FROM companies	100 row(s) returned	0.00
4	11:51:03	SELECT MIN(company_id), MAX(company_id) FROM companies	1 row(s) returned	0.00
5	11:51:06	SELECT * FROM companies WHERE company_name IS NULL	0 row(s) returned	0.00



```
210  -- Transformation
211  ALTER TABLE credit_cards
212  ADD PRIMARY KEY (id)
213  ;
```

100% 2:213

Action Output

	Time	Action	Response
1	12:19:32	ALTER TABLE credit_cards ADD PRIMARY KEY (id)	0 row(s) a

- The expiration date provided in the dataset is not in standard SQL date format. Therefore, the expiring_date column is initially defined as VARCHAR(10) and later converted to a DATE type using STR_TO_DATE.
  The original format is (MM/DD/YY), which was validated by detecting the leap year 2028 (02/29/28).
  To ensure data integrity, an intermediate column (expiring_date_temp) will be used during the conversion process and then renamed as the final expiring_date column.

```sql
-- Expiring date normalization
ALTER TABLE credit_cards
ADD expiring_date_temp DATE
;
-- Data checking in order to erase original column and rename the new one
UPDATE credit_cards
SET expiring_date_temp = STR_TO_DATE(expiring_date, '%m/%d/%Y')
WHERE expiring_date IS NOT NULL
LIMIT 100000
;
SELECT expiring_date, expiring_date_temp   -- Check
FROM credit_cards
LIMIT 10
;
ALTER TABLE credit_cards        -- Drop the column
DROP COLUMN expiring_date
;
ALTER TABLE credit_cards        -- and rename
RENAME COLUMN expiring_date_temp TO expiring_date
;
```

100% 2:233

Action Output

	Time	Action	Response	Duration
1	12:33:39	ALTER TABLE credit_cards ADD expiring_date_temp DATE	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.045 sec
2	12:33:44	UPDATE credit_cards SET expiring_date_temp = STR_TO_DATE(expiring...	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0	0.062 sec
3	12:33:47	SELECT expiring_date, expiring_date_temp -- Check FROM credit_cards...	10 row(s) returned	0.00080
4	12:34:10	ALTER TABLE credit_cards -- Drop the column DROP COLUMN expiring...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.029 sec
5	12:34:21	ALTER TABLE credit_cards -- and rename RENAME COLUMN expiring_dat...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec

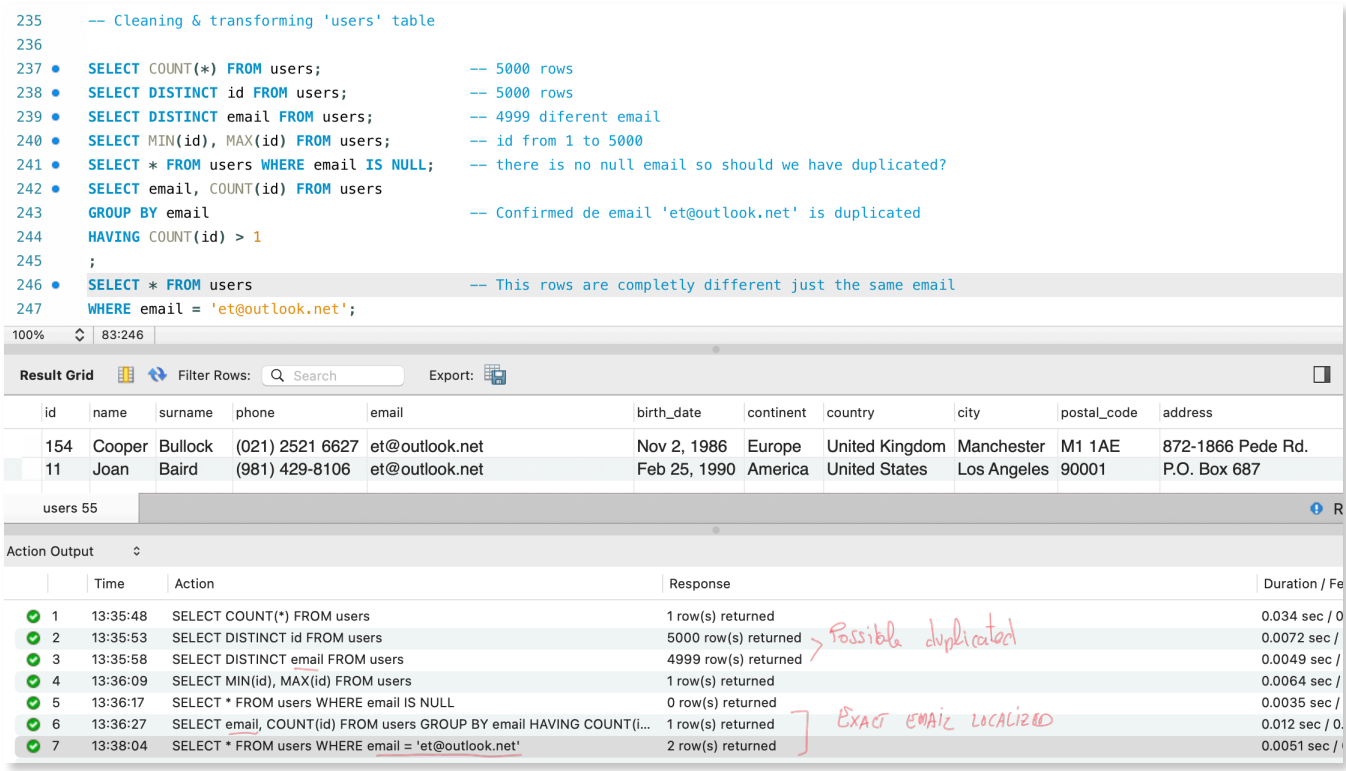
ORIGINAL → TRANSFORMED

Result Grid Filter Rows: Search Export:

expiring_date	expiring_date_temp
09/27/25	2025-09-27
12/28/28	2028-12-28
11/26/26	2026-11-26
07/27/27	2027-07-27
04/25/26	2026-04-25
11/27/26	2026-11-27
12/27/29	2029-12-27
02/28/26	2026-02-28
11/25/28	2028-11-25
02/28/25	2025-02-28

‘users’ table:

- Primary Key —> ‘id’.
- It is not necessary to change types of the PK (id) or the rest of the fields except 'birth_date' that I want to convert in DATE datatype .
- Using EDA analysis, I have verified that this table does not have duplicates in the 'id' so it will not give us problems as PK also the 'id' has correlative numbers from 1 to 5000.
- Also in the analysis I have discovered that even if there are no completely duplicate rows there are 2 records with the same email, in principle it does not give us a bigger problem, except if we had to do operations with the email or email marketing campaigns. These two records only coincide with the email which could be due to a typing error in the registration by the user.



```
-- Cleaning & transforming 'users' table

SELECT COUNT(*) FROM users;                -- 5000 rows
SELECT DISTINCT id FROM users;             -- 5000 rows
SELECT DISTINCT email FROM users;          -- 4999 diferent email
SELECT MIN(id), MAX(id) FROM users;        -- id from 1 to 5000
SELECT * FROM users WHERE email IS NULL;   -- there is no null email so should we have duplicated?
SELECT email, COUNT(id) FROM users
GROUP BY email                             -- Confirmed de email 'et@outlook.net' is duplicated
HAVING COUNT(id) > 1
;
SELECT * FROM users                        -- This rows are completly different just the same email
WHERE email = 'et@outlook.net';
```

100% 83:246

Result Grid Filter Rows: Search Export:

id	name	surname	phone	email	birth_date	continent	country	city	postal_code	address
154	Cooper	Bullock	(021) 2521 6627	et@outlook.net	Nov 2, 1986	Europe	United Kingdom	Manchester	M1 1AE	872-1866 Pede Rd.
11	Joan	Baird	(981) 429-8106	et@outlook.net	Feb 25, 1990	America	United States	Los Angeles	90001	P.O. Box 687

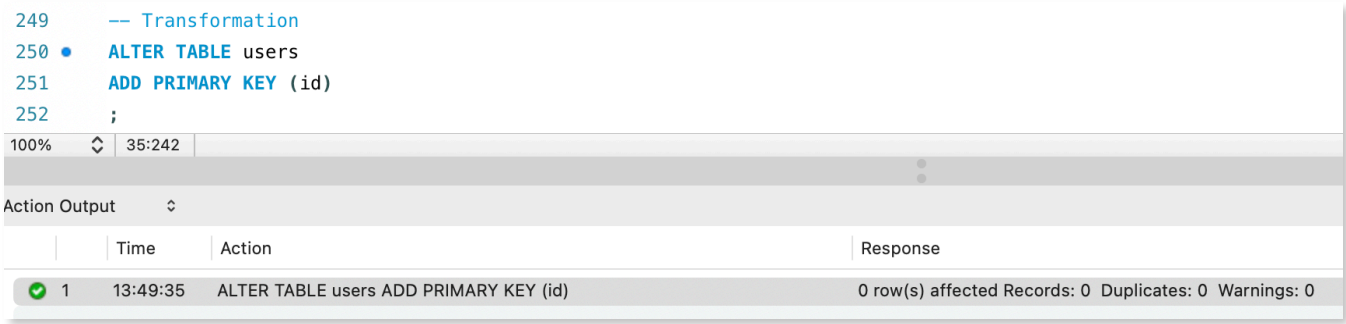
users 55

Action Output

	Time	Action	Response	Duration / Fe
1	13:35:48	SELECT COUNT(*) FROM users	1 row(s) returned	0.034 sec / 0
2	13:35:53	SELECT DISTINCT id FROM users	5000 row(s) returned	0.0072 sec /
3	13:35:58	SELECT DISTINCT email FROM users	4999 row(s) returned	0.0049 sec /
4	13:36:09	SELECT MIN(id), MAX(id) FROM users	1 row(s) returned	0.0064 sec /
5	13:36:17	SELECT * FROM users WHERE email IS NULL	0 row(s) returned	0.0035 sec /
6	13:36:27	SELECT email, COUNT(id) FROM users GROUP BY email HAVING COUNT(i...	1 row(s) returned	0.012 sec / 0.
7	13:38:04	SELECT * FROM users WHERE email = 'et@outlook.net'	2 row(s) returned	0.0051 sec /

Possible duplicated

EXACT EMAIL LOCALIZED



```
-- Transformation
ALTER TABLE users
ADD PRIMARY KEY (id)
;
```

100% 35:242

Action Output

	Time	Action	Response
1	13:49:35	ALTER TABLE users ADD PRIMARY KEY (id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

- The ‘birth_date’ provided in the CSV is not in standard SQL date format. Therefore, the ‘birth_date’ column is initially defined as VARCHAR(100) to import and now I convert it to a DATE datatype using STR_TO_DATE.

  To ensure data integrity, an intermediate column (birth_date_temp) will be used during the conversion process and then renamed as the final birth_date column.

```
253     -- 'birth_date' normalization
254 •   ALTER TABLE users
255     ADD birth_date_temp DATE
256     ;
257     -- Data checking in order to erase original column and rename the new one
258 •   UPDATE users
259     SET birth_date_temp = STR_TO_DATE(birth_date, '%b %e, %Y')
260     WHERE birth_date IS NOT NULL
261     LIMIT 60000
262     ;
263 •   SELECT birth_date, birth_date_temp   -- Check
264     FROM users
265     LIMIT 10
266     ;
267 •   ALTER TABLE users           -- Drop the column
268     DROP COLUMN birth_date
269     ;
270 •   ALTER TABLE users       -- and rename
271     RENAME COLUMN birth_date_temp TO birth_date
272     ;
```

100% 32:247

Action Output

	Time	Action	Response
1	14:32:20	ALTER TABLE users ADD birth_date_temp DATE	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
2	14:32:24	UPDATE users SET birth_date_temp = STR_TO_DATE(birth_date, '%b %e, %Y') WHERE birth...	5000 row(s) affected Rows matched: 5000 Changed: 5000 W
3	14:32:31	SELECT birth_date, birth_date_temp -- Check FROM users LIMIT 10	10 row(s) returned
4	14:32:39	ALTER TABLE users -- Drop the column DROP COLUMN birth_date	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
5	14:32:46	ALTER TABLE users -- and rename RENAME COLUMN birth_date_temp TO birth_date	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

Result Grid Filter Rows: Search Export:

birth_date	birth_date_temp
Nov 17, 1985	1985-11-17
Aug 23, 1992	1992-08-23
Apr 29, 1998	1998-04-29
Feb 18, 1989	1989-02-18
Sep 26, 1998	1998-09-26
Oct 15, 1989	1989-10-15
Dec 4, 1981	1981-12-04
Aug 1, 1993	1993-08-01
Jan 24, 1987	1987-01-24
Apr 30, 1984	1984-04-30

‘transactions’ table:

- Primary Key —> 'id'.
- It is not necessary to change types of the PK (id), but I am going to change types in many columns since they are FK and they have to be the same as their PK.
- Using a basic EDA, I have verified that this table has no duplicates or nulls in its 100,000 rows. I have also specifically probed that the columns that are going to be the FKs had no problems.

```
274    -- Cleaning & transforming 'transactions' table
275
276    SELECT COUNT(*) FROM transactions;                    -- 100000 rows
277    SELECT DISTINCT id FROM transactions;                 -- 100000 rows
278    SELECT DISTINCT card_id FROM transactions;            -- 50000 all card ID has been utilizated at list once finished or not transaction
279    SELECT DISTINCT business_id FROM transactions;        -- 100 all companies has transactions, finished or not
280    SELECT DISTINCT user_id FROM transactions;            -- 5000 all users has transactions, finished or not
281    SELECT * FROM transactions WHERE id IS NULL;
282    SELECT * FROM transactions WHERE `timestamp` IS NULL;
283    SELECT * FROM transactions WHERE amount IS NULL;
284    SELECT * FROM transactions WHERE declined IS NULL;    -- Ihaven't detect any NULL valeu in all the checked column.
```

100% 52:284

Action Output

	Time	Action	Response	Duration / Fetch Tim
1	15:07:11	SELECT COUNT(*) FROM transactions	1 row(s) returned	0.045 sec / 0.00067
2	15:07:16	SELECT DISTINCT id FROM transactions	100000 row(s) returned	0.065 sec / 0.011 sec
3	15:07:22	SELECT DISTINCT card_id FROM transactions	5000 row(s) returned	0.046 sec / 0.00054
4	15:07:26	SELECT DISTINCT business_id FROM transactions	100 row(s) returned	0.044 sec / 0.00001(
5	15:07:33	SELECT DISTINCT user_id FROM transactions	5000 row(s) returned	0.043 sec / 0.00046
6	15:07:37	SELECT * FROM transactions WHERE id IS NULL	0 row(s) returned	0.051 sec / 0.000012
7	15:07:40	SELECT * FROM transactions WHERE `timestamp` IS NULL	0 row(s) returned	0.056 sec / 0.00001
8	15:07:42	SELECT * FROM transactions WHERE amount IS NULL	0 row(s) returned	0.051 sec / 0.000013
9	15:07:45	SELECT * FROM transactions WHERE declined IS NULL	0 row(s) returned	0.050 sec / 0.00000

```
286    -- Transformation
287    ALTER TABLE transactions
288    ADD PRIMARY KEY (id)
289    ;
```

100% 25:284

Action Output

	Time	Action	Response
1	15:09:39	ALTER TABLE transactions ADD PRIMARY KEY (id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

- Data transformation:
- Everything is OK except column 'business_id' that I will change name and 5 columns to which I will change DataType:

- business_id —> VARCHAR(15)
- timestamp. —> TIMESTAMP
- amount — > DECIMAL(10, 2)
- lat —> DECIMAL(20, 16)
- longitude —> DECIMAL(20, 16)

```
291    -- change name and datatype  column 'business_id'
292    ALTER TABLE transactions
293    MODIFY COLUMN business_id VARCHAR(15)
294    ;
295    ALTER TABLE transactions
296    RENAME COLUMN business_id TO company_id
297    ;
298    ALTER TABLE transactions
299    MODIFY COLUMN `timestamp` TIMESTAMP
300    ;
301    ALTER TABLE transactions
302    MODIFY COLUMN amount DECIMAL(10, 2)
303    ;
304    ALTER TABLE transactions
305    MODIFY COLUMN lat DECIMAL(20, 16)
306    ;
307    ALTER TABLE transactions
308    MODIFY COLUMN longitude DECIMAL(20, 16)
309    ;
```

100% 18:286

Action Output

	Time	Action	Response
1	19:31:40	ALTER TABLE transactions MODIFY COLUMN business_id VARCHAR(15)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
2	19:32:10	ALTER TABLE transactions RENAME COLUMN business_id TO company_id	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
3	19:42:23	ALTER TABLE transactions MODIFY COLUMN `timestamp` TIMESTAMP	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
4	19:43:46	ALTER TABLE transactions MODIFY COLUMN amount DECIMAL(10, 2)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
5	20:02:31	ALTER TABLE transactions MODIFY COLUMN lat DECIMAL(20, 16)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
6	20:02:57	ALTER TABLE transactions MODIFY COLUMN longitude DECIMAL(20, 16)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

Step 5 - Data modeling & star schema implementation

Now that we have all table & data cleaned and well organized is the moment to create the relationship between tables.

We have been asked to create a star design, that's why the FKs are established in the ‘transactions’, the fact table, and for the moment I will only relate 3 of the 4 dimension tables in a star scheme, because in another exercise we will be asked to create the 'products' relation.

```
310    -- DATA MODELING
311
312    -- FOREIGN KEY (FK) Creation
313
314    ALTER TABLE transactions
315    ADD CONSTRAINT FK_CreditCardsTransactions FOREIGN KEY (card_id)
316    REFERENCES credit_cards(id)
317    ;
318    ALTER TABLE transactions
319    ADD CONSTRAINT FK_CompaniesTransactions FOREIGN KEY (company_id)
320    REFERENCES companies(company_id)
321    ;
322    ALTER TABLE transactions
323    ADD CONSTRAINT FK_UsersTransactions FOREIGN KEY (user_id)
324    REFERENCES users(id)
325    ;
```

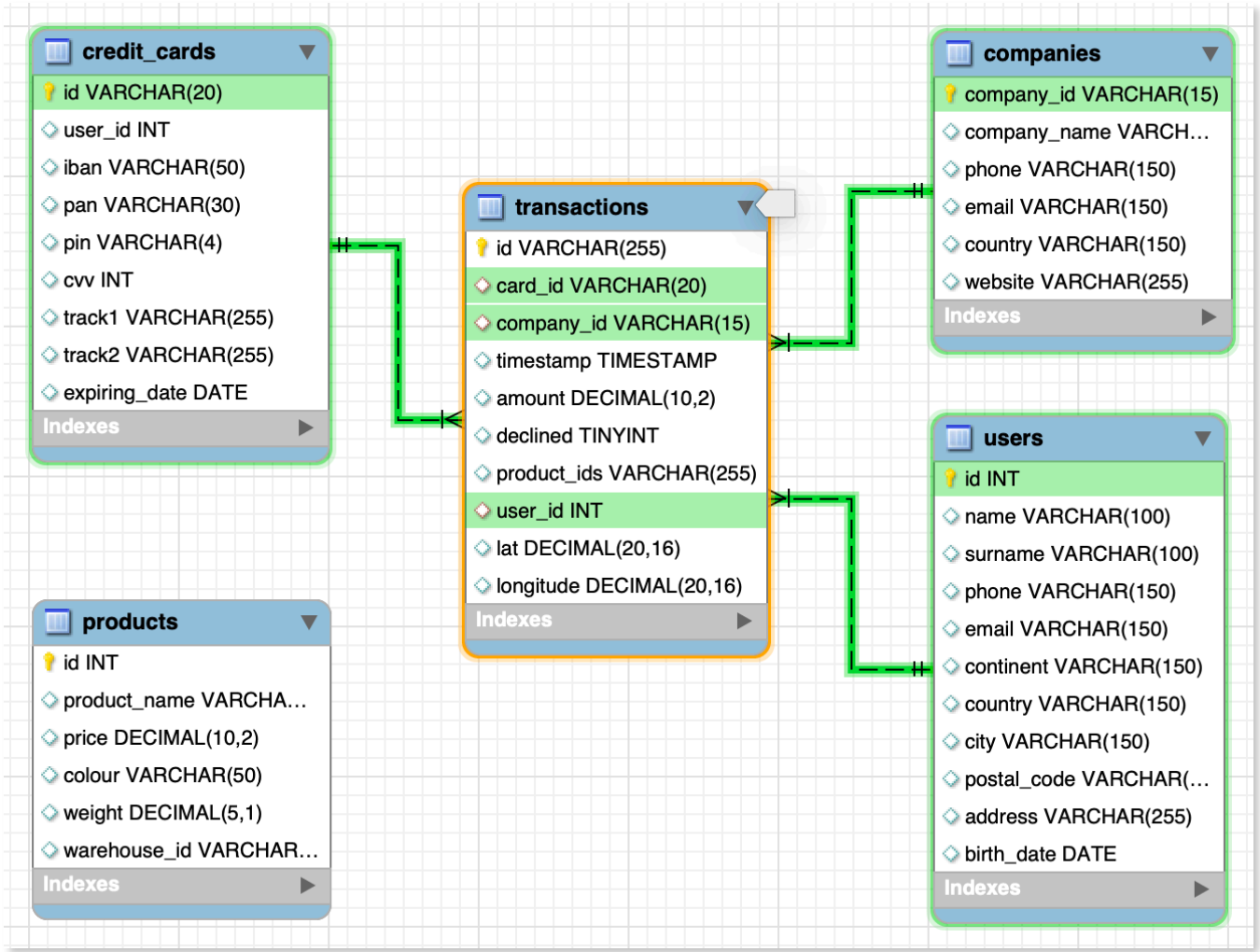
100% 25:305

Action Output

	Time	Action	Response
1	21:30:19	ALTER TABLE transactions ADD CONSTRAINT FK_CreditCardsTransactions FOREIGN K...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
2	21:30:24	ALTER TABLE transactions ADD CONSTRAINT FK_CompaniesTransactions FOREIGN K...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
3	21:30:28	ALTER TABLE transactions ADD CONSTRAINT FK_UsersTransactions FOREIGN KEY (u...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

Star schema design:

- As requested in the statement after the whole process, table creation, data import, data cleaning and transformation and the creation of the Foreign Key, we obtain the DB model required in the statement:
- Dimensional Model in Star. (OK)
- Use at least 4 tables (OK)
- That allows you to ask the questions of Exercise 1 and 2. (OK)



- Exercise 1

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

Subquery using at list 2 tables:

- I perform a subquery of the 'transactions' table counting the number of transactions for each 'user_id' and filtering with HAVING, that shows only the 'user_id' with more than 80 'num_transactions'.
- I get the result (in this case 3 'user_id') shown in the table (alias) 'transaction_user' and make a JOIN (INNER JOIN) with the table 'users' .
- The JOIN only returns me rows that are in the two tables, in this case 3 ‘user_id'.
- I show the desired fields of the JOIN sorted descending by 'total_transactions.

```sql
SELECT  u.id,
        u.`name`,
        u.surname,
        transaction_user.num_transactions AS total_transactions
FROM users u
JOIN (
    SELECT  t.user_id,
            COUNT(t.id) AS num_transactions
    FROM transactions t
    WHERE declined = 0
    GROUP BY t.user_id
    HAVING num_transactions > 80
) AS transaction_user
ON u.id = transaction_user.user_id
ORDER BY total_transactions DESC
;
```

100% 4:328

Result Grid Filter Rows: Search Export:

id	name	surname	total_transactions
185	Molly	Gilliam	107
289	Dxwgi	Hwcru	91
318	Bnyr	Astuw	86

Result 87

Action Output

	Time	Action	Response
1	11:47:34	SELECT u.id, u.`name`, u.surname, transaction_user.num_transaction...	3 row(s) returned

- Exercise 2

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

- As the opposite is not requested, I will use a JOIN of 3 tables to have access to all the necessary fields.
- I apply WHERE filter for the requested company.
- I make a GROUP BY grouping by iban and use the AVG aggregator to calculate the average by iban.
- Finally I present the requested data, average of 'amount' grouped by iban and only those of the company Donec Ltd.
- I have ordered the presentation descending by average ‘amount’.
- Relevant tips:
  - SELECT by c.company_id —> protection agains 2 companies with the same name.
  - Filter ’t.declined’ = 0 — > I consider that we want to know the amount_average of the transactions ended OK!

```sql
SELECT  cc.iban,
        c.company_id,
        c.company_name,
        ROUND(AVG(t.amount),2) AS amount_average
FROM transactions t
JOIN credit_cards cc ON t.card_id = cc.id
JOIN companies c ON t.company_id = c.company_id
WHERE c.company_name = 'Donec Ltd'
AND t.declined = 0                          -- Transactions finished ok
GROUP BY cc.iban, c.company_name,c.company_id      -- company_id to avoid problems ex: 2 companies same name
ORDER BY AVG(t.amount) DESC
;
```

100% 3:353

Result Grid Filter Rows: Search Export:

iban	company_id	company_name	amount_average
XX3830178139196201993663 52	b-2242	Donec Ltd	680.69
XX6377063573975703949739 13	b-2242	Donec Ltd	680.01
XX9713939714652922023122 59	b-2242	Donec Ltd	645.46
XX1718471169288923759693 07	b-2242	Donec Ltd	628.89
XX2254246388185424062235 75	b-2242	Donec Ltd	608.68
XX7488907290571957117660 71	b-2242	Donec Ltd	607.29

Result 7

Action Output

	Time	Action	Response
1	12:32:23	SELECT cc.iban, c.company_id, c.company_name, ROUND(AVG(t.amount),2) AS amount_average FROM t...	370 row(s) returned

Level 2

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:

For this task, we're working with 5000 cards in the 'credit_cards table'. To make our code run smoothly, I'll assume all the cards are currently active. I'll use my queries to figure out which cards need to be turned off. I'll break my code into three steps:

Step 1: Table creation and initial data insertion.

- Creation of the 'credit_card_activated' table that would be the status table.
- This table has the 'card_id' column as PK and where I'll store the 'id' of the 'credit_cards' table.
- I've created the 'activated' field, I define this field as a TINYINT to minimize resource consumption. I will store (1) if it is an active card and (0) if it is a deactivated card.
- Through the INSERT instruction I incorporate the data into the status table, on the one hand the 'card_id' from the table 'credit_cards' and the column 'activated' I fill it all with 1 (activated). Total 5000 active cards.

Step 2: Query or filter to find cards to deactivate.

- At this point what I want is to find out if the last 3 transactions of each card used, have been rejected ('declined' = 1) or approved ('declined' = 0).
- To elaborate this query I use CTEs (WITH) that will help me simplify the structure and reading of the code:

- 'Last_transactions_card' with this query and using a window function (ROW_NUMBER) I've managed to sort the transactions grouped by card (PARTITION BY) and ordered DESC by 'timestamp' from the newest transaction to the oldest. ROW_NUMBER lists me inside each card in the order we have applied to it.

- 'cards_three_declined' mediante esta consulta encuentro las tarjetas que tienen 3 transacciones 'declined' = 1 dentro de sus tres últimas transacciones.

Step 3: Update and change the status of the cards to be deactivated

- Using the UPDATE and SET instruction, I change the state of the 'activated' column of the 'credit_card_activated' table to (activated = 0), but with the WHERE clause... IN, I only change the state of the selected cards using the query 'cards_three_declined'.

Final comments:

About this exercise, for me it is important to highlight what I learned about CTE's and Window Functions.

On the other hand, I want to emphasize that I think I have given the correct approach by inserting, at the beginning of the whole process, in the 'credit_card_activated' status table, all the cards from the 'credit_card' table and leaving them activated. It would have been a mistake to take only the cards in the transactions table, since there could be cards in 'credit_cards' that did not have transactions and that therefore we would never consider them activated.

Extra:

Just letting you know, my query is all set to address the question from the exercise. However, we could give it a little extra flair to make it more dynamic. If we're up for it, I'd keep an eye on any new transactions in the 'transactions' table and update the status of a deactivated card to reactivate it if needed. I'd change the 'activated' column values from (0 deactivated) to (1 activated).

I'd use the 'case' instruction to re-activate a card if, after running the new Step 2 queries, this card has fewer than 3 declined operations from its last 3 transactions.

```sql
CREATE TABLE IF NOT EXISTS credit_card_activated (
    card_id VARCHAR(20) PRIMARY KEY,                    -- Status table creation to store the status
    activated TINYINT
);

INSERT INTO credit_card_activated
    (card_id, activated)                                -- Insert initial values into table (5000 rows)
    SELECT cc.id, 1 FROM credit_cards cc                -- All the Credit cards from credit_card table
    WHERE cc.id = cc.id                                 -- I fill all 'activated' with 1 -- > activated
;

WITH                                                    -- CTEs for code simplification
    last_transactions_card AS (                         -- Find transactions by card
    SELECT  t.card_id,                                  -- With ROW_NUMBER I can know card_position by card
            t.`timestamp`,
            t.declined,
            ROW_NUMBER () OVER (PARTITION BY t.card_id ORDER BY t.`timestamp` DESC) AS card_position
    FROM transactions t),

    cards_three_declined AS (                           -- Find cards thatn have 3 declined
    SELECT  card_id,
            SUM(declined) AS total_declined
    FROM last_transactions_card
    WHERE card_position <= 3                            -- in their 3 last transaction
    GROUP BY card_id
    HAVING total_declined = 3)

UPDATE credit_card_activated cca                        -- Update statement setting 'activated' to 0
SET                                                     -- just for the 'card_id' with 3 declined
    activated = 0                                       -- in its last 3 transactions
WHERE cca.card_id IN (SELECT card_id FROM cards_three_declined)
AND cca.card_id IS NOT NULL                             -- 2 lines necessaries to bypass
LIMIT 40000                                             -- the MySQL Workbench safe mode
;
```

100% 56:415

Action Output

	Time	Action	Response
1	21:42:32	CREATE TABLE IF NOT EXISTS credit_card_activated (card_id VAR...	0 row(s) affected
2	21:42:58	INSERT INTO credit_card_activated (card_id, activated) -- Inse...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
3	21:43:48	WITH -- CTEs for code simplification last_transactions_card...	5 row(s) affected Rows matched: 5 Changed: 5 Warnings: 0

- Exercise 1

Quantes targetes estan actives?

Active cards = There are 4995 active cards

After having created the 'credit_card_activated' table it is very easy to find out how many cards are active at the time of analysis. It is enough to count the rows with ('activated' = 1) and we obtain the number of credit cards that are active.

```
414 •   SELECT COUNT(*) AS num_active_cards
415     FROM credit_card_activated
416     WHERE activated = 1
417     ;
```

100% 1:408

Result Grid Filter Rows: Search Export:

num_active_cards
4995

Result 6

Action Output

	Time	Action
1	12:22:06	SELECT COUNT(*) AS num_active_cards FROM credit_card_activated WHERE activated = 1

Level 3

Crea una taula amb la qual puguem unir les dades del nou arxiu products.csv amb la base de dades creada, tenint en compte que des de transaction tens product_ids. Genera la següent consulta:

Creation and filling of the table

The relationship between the 'transactions' and 'products' tables is many to many (n:m), a transaction can contain several products and on the other hand a product can appear in several transactions. Because of this we cannot establish a direct relationship between the two tables and we must create an intermediate table 'transaction_product' that transforms this relationship from many to many, to a relationship from one 'products' to many 'transaction_product' and from many to one between 'transaction_product' and 'transactions'.

```
429  CREATE TABLE transaction_product (
430      transaction_product_id INT AUTO_INCREMENT PRIMARY KEY,
431      transaction_id VARCHAR(255),
432      product_id INT,
433      FOREIGN KEY (transaction_id) REFERENCES transactions(id),
434      FOREIGN KEY (product_id) REFERENCES products(id),
435      UNIQUE (transaction_id, product_id)
436  );
```

100% 2:418

Action Output

	Time	Action	Response
1	22:32:13	CREATE TABLE transaction_product (transaction_product_id INT AUTO_INCREME...	0 row(s) affected

In the creation of the 'transaction_product' table I have defined:

- A field 'transaction_product_id' INT AUTO_INCREMENT as PK.
- 'Transaction_id' : which will include the PK 'id' of the table 'transactions' will be repeated as many times the value of the field as the number of products includes the transaction.
- 'Product_id': which will include the result of the extraction of the different 'products_id' that are separated by commas in the 'product_ids' column of the 'transactions' table
- At this moment I also create the relationships with the FK
- Finally and **very important to declare UNIQUE** the combination (transaction_id & product_id)

The biggest hurdle here is getting the data from the ‘product_ids’ column. I was thinking of using the JSON_TABLE function to handle this. It makes a table from a JSON array and splits the ‘product_id’ by rows, which should match up with the ‘product_ids’ field in the ‘transactions’ table.

Since we don’t have the JSON array in the ‘product_ids’ field to begin with, we’ll need to create it. Here’s how I’ve done it:

- First, clear out any extra spaces after each comma or anywhere in the ‘product_ids’ field using the REPLACE string function.
- Then, to get the array, I use CONCAT “[“ to start my JSON array.

Once we have the array, I use the JSON_TABLE function to extract the data and assign each ‘product_id’ to its ‘transaction_id’.

Finally, I use the “INSERT INTO SELECT” statement to insert the SELECT ‘transaction_id’ & ‘product_id’ into the intermediate table named ‘transaction_product’.

With this table created and filled, we’ll be all set to tackle Exercise 1 of this Level 3!

```
439  INSERT INTO transaction_product (transaction_id, product_id)
440  SELECT  t.id AS transaction_id,
441          jt.product_id
442  FROM transactions t,
443  JSON_TABLE(
444      CONCAT('[', REPLACE(product_ids, ' ', ''), ']'),
445      '$[*]' COLUMNS (product_id INT PATH '$')
446  ) AS jt
447  ;
```

100% 1:429

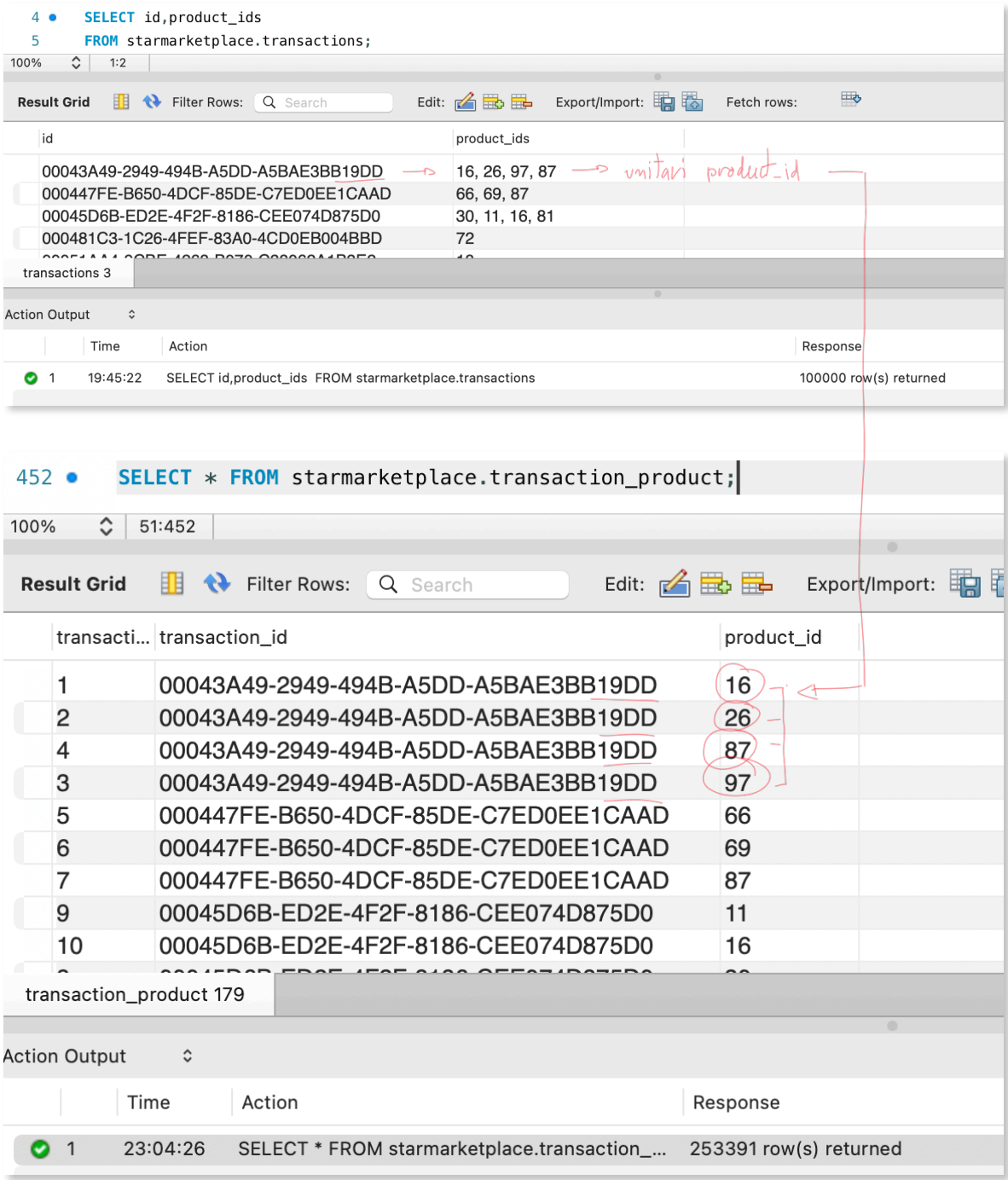
Action Output

	Time	Action	Response
1	23:00:13	INSERT INTO transaction_product (transaction...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0

Verification: For verification and information purposes only. SELECT * FROM 'transaction_product' shows 253391 rows, not 100000, when we consider the product details of each transaction.

Visual inspection of the first table elements reveals that the 'transaction_id' repeats as many times as products in the original 'product_ids' field. Each row contains one of the 'product_ids' from the 'transactions' table.

The visual check confirms the correctness of the result.



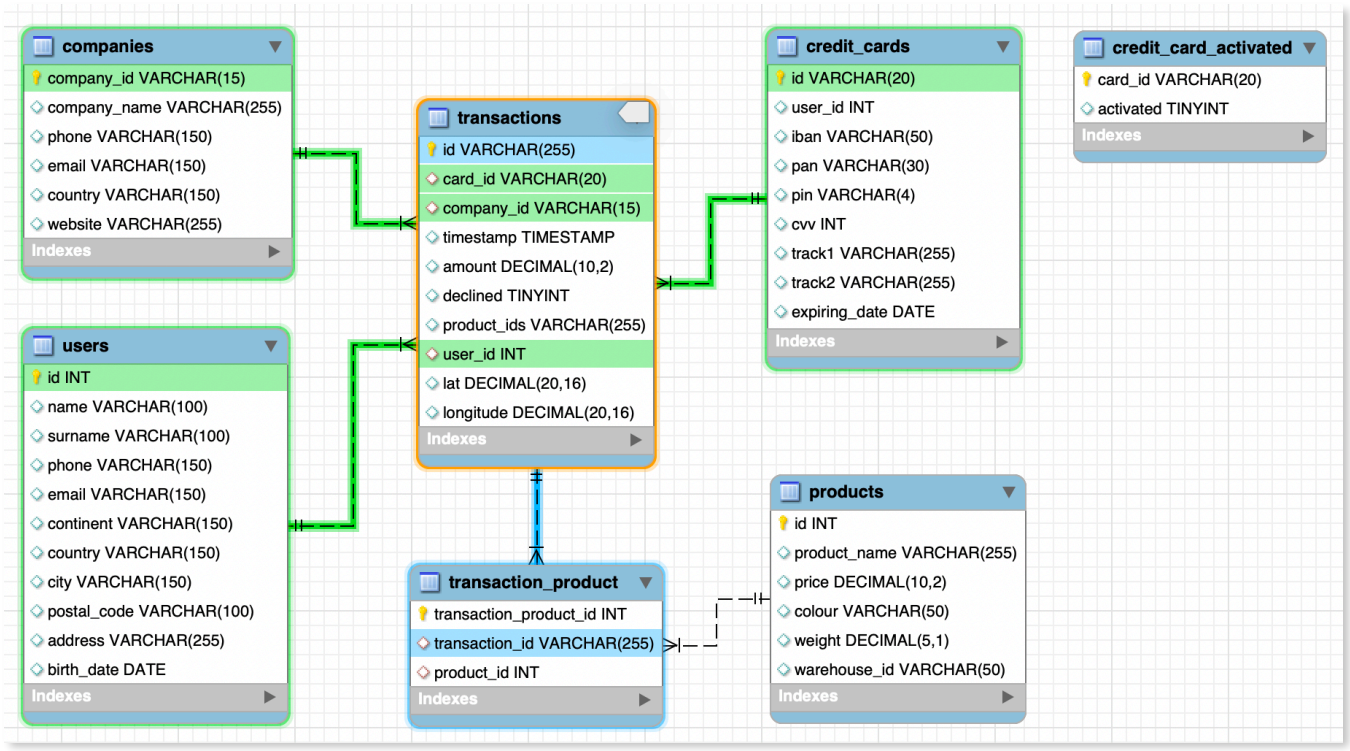
The model:

At the end, after reviewing the model following all transformations and integrations, I can confirm that the resulting model remains a Star Schema.

This is because the intermediate table ‘transaction_product’, placed between the fact table ‘transactions’ and the dimension table ‘products’, is purely auxiliary. It solves a specific problem but does not provide any dimensional value itself.

On the other hand, the table created at Level 2, ‘credit_card_activated’, has a 1:1 relationship with ‘credit_cards’, but declaring it is unnecessary. This table is not the best option for storing a card’s status. One alternative would be to add it as a field in ‘credit_cards’, but this value is expected to change frequently, making it unsuitable for a dimension table.

In conclusion, from my point of view, the card status should be treated as a calculated field, which could fit perfectly within a table_view.



- Exercise 1

Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

To find the number of times that each product has been sold I have done it working on the new table 'transaction_product' and through JOINS with the tables 'products' and 'transactions' I've obtained the values that interest us for the SELECT.
At the end, to be able to give an answer is no more than a query with GROUP BY per product and a COUNT of transactions.

Option A:

```
453  SELECT  p.id,
454          p.product_name AS product,
455          COUNT(tp.transaction_id) AS sales_qty
456  FROM transaction_product tp
457  JOIN transactions t ON tp.transaction_id = t.id
458  JOIN products p ON tp.product_id = p.id
459  WHERE t.declined = 0
460  GROUP BY tp.product_id
461  ORDER BY COUNT(tp.transaction_id) DESC
462  ;
```

100% 14:453

Result Grid Filter Rows: Search Export:

id	product	sales_qty
52	riverlands the duel	2642
29	Tully maester Tarly	2627
21	duel Direwolf	2603
16	the duel warden	2602

Result 9

Action Output

	Time	Action	Response
1	13:13:34	SELECT p.id, p.product_name AS product, COUNT(tp.transaction_id) AS sales_qty FRO...	100 row(s) returned

At the end I have made another query, but this time on the original table 'transactions', as you can see, the result is exactly the same. Before making the breakdown of products by transaction, we could not have calculated the number of sales of each product either with option A or with B. This option B clearly describes the relationship that has been established between the three tables.

Option B:

```
465 •  SELECT  p.id,
466            p.product_name AS product,
467            COUNT(t.id) AS sales_qty
468    FROM transactions t
469    JOIN transaction_product tp ON tp.transaction_id = t.id
470    JOIN products p ON tp.product_id = p.id
471    WHERE t.declined = 0
472    GROUP BY p.id
473    ORDER BY COUNT(t.id) DESC
474    ;
```

100% 39:461

Result Grid Filter Rows: Search Export:

id	product	sales_qty
52	riverlands the duel	2642
29	Tully maester Tarly	2627
21	duel Direwolf	2603
16	the duel warden	2602

Result 11

Action Output

	Time	Action	Response
1	20:40:03	SELECT p.id, p.product_name AS product, COUNT(t.id) AS sales_qty FROM transactions t J...	100 row(s) returned